

How Consistent Are LLM Agents?

Measuring Behavioral Reproducibility in
Multi-Step Tool-Calling Pipelines

Abel Yagubyan
Independent Researcher
abelyagubyan@berkeley.edu

April 2026

Abstract

Large language model (LLM) agents with tool-calling capabilities are increasingly deployed in production systems, raising a fundamental reliability question: *does the same agent behave the same way twice?* Prior work has established that repeated-trial success is unstable (τ -bench’s $\text{pass}^{\wedge}k$); we characterize *where* in the pipeline the instability lives. We present a systematic empirical study of **behavioral consistency in multi-step tool-calling agents**, measuring whether agents select the same tools, in the same order, with the same arguments, across repeated identical invocations. Unlike prior work on consistency in ReAct-style agents (search-only, free-text actions), we study the richer setting of *structured tool-calling interfaces* with typed parameters and consequential side effects.

Using a benchmark of 19 tasks across five categories (data retrieval, scheduling, computation, multi-tool composition, and ambiguous requests), we evaluate six models from three providers (OpenAI, Anthropic, Meta) across 1,140 agent traces, collected in July 2026 through a single provider-normalized API with version-pinned model identifiers, with **every raw trace released**. This paper’s results are additionally a **replication**: an earlier (late-2025) collection with the same harness produced aggregates that our fresh collection reproduces almost exactly, and where the two collections disagree we say so explicitly.

We identify a “**structural consistency, parametric variance**” pattern: agents reliably select the same tools in the same order (mean TSS = 0.88, 95 % CI [0.85, 0.91]) but vary substantially in the arguments they provide (mean AC = 0.70, [0.65, 0.75]); the gap is large (Cohen’s $d = 0.74$, $p \approx 10^{-12}$, with clustering caveats in Section 4.7) and **replicates across both collections to within one point**. Robust secondary findings: **62 % of behavioral divergence** originates in the first two pipeline steps, and **natural language outputs almost never match** (4 % exact) even when tool sequences are identical. Two earlier secondary claims *weakened* under replication and we report them at their replicated strength: ambiguous task specifications reduce argument consistency by 13% (directionally consistent but not significant, $d = 0.33$, $p = 0.16$), and cross-model differences are marginal for both metrics ($\eta^2 \approx 0.10$). The earlier snapshot’s correctness–consistency association also proved generation-dependent: on fresh traces from mid-2026 models, which are markedly more consistent (median condition-level TSS = 1.0), the association shrinks to a weak rank correlation ($\rho = 0.22$, $p = 0.017$; Pearson n.s.), so we present TSS not as a correctness proxy but as a variance signal whose predictive value declines as models grow more consistent. Model-level consistency itself improved substantially between the two collections, a longitudinal observation about model progress that, to our knowledge, is new.

We release all code, benchmark definitions, raw traces (1,140), per-condition data, and analysis scripts at <https://github.com/Abelo9996/agent-consistency>.

1 Introduction

The deployment of LLM-based agents with tool-calling capabilities has accelerated rapidly, with production systems now using agents to search databases, send emails, manage calendars, and orchestrate multi-step workflows through structured function calls (Qin et al., 2024; Li et al., 2023; Schick et al., 2023). As these systems mature, a reliability question that has received surprisingly little systematic attention comes to the fore: *if you run the same agent on the same task twice, do you get the same behavior?*

This question has immediate practical consequences:

- **Testing.** If agents behave differently across runs, unit tests asserting on outputs are flaky by design. Reliable behavioral invariants are needed to test that an agent “does the right thing” (Kapoor and Narayanan, 2024).
- **Debugging.** Failure reproducibility is a prerequisite for root-cause analysis. Behavioral variance makes production failures intermittent and difficult to trace.
- **Safety and auditability.** High-stakes deployments require guarantees that the agent will not take an unexpected action on a re-run (e.g., sending duplicate emails, creating conflicting calendar events) (Weidinger et al., 2021).
- **Cost optimization.** If behavioral variance is predictable from task characteristics, consistency-aware routing can assign tasks to cheaper models when variance is tolerable and reserve high-consistency models for critical workflows.

The gap in prior work. Mehta et al. (2026) studied behavioral consistency in ReAct-style agents (Yao et al., 2023) on HotpotQA, finding that agents produce 2.0–4.2 distinct action sequences per 10 runs and that inconsistency predicts failure. This is important but limited to *search-only* actions in a question-answering setting. Real-world agents operate over *structured tool-calling interfaces* with typed parameters, multiple heterogeneous tools, and multi-step pipelines. The distinction matters: tool calls are discrete typed objects (not free-form text), have observable side effects (emails sent, events created), and compose in sequences where early divergence propagates through subsequent steps. It is unclear whether consistency patterns from ReAct agents transfer to this richer action space.

This work. We extend the study of behavioral consistency to multi-step tool-calling agents across diverse task types, making the following contributions:

1. **Benchmark.** 19 tasks spanning five categories, paired with 10 deterministic simulated tools that isolate LLM variance from environmental non-determinism (Section 4.1).
2. **Formal metric framework.** Formal definitions of Tool Sequence Similarity (TSS), Argument Consistency (AC), divergence point, and output agreement, targeting distinct behavioral layers (Section 3.2).
3. **The structural/parametric distinction.** $TSS = 0.88$ substantially exceeds $AC = 0.70$ across all models and categories ($d = 0.74$, $p \approx 10^{-12}$, replicated across two collections eight months apart); this pattern is not captured by prior single-metric consistency studies (Section 5.1).
4. **Correctness analysis with an honest replication arc.** On the late-2025 collection TSS predicted rubric correctness strongly ($d = 0.81$); on the fresh 2026 collection, where models are far more consistent, only a weak rank association survives ($\rho = 0.22$, $p = 0.017$). We report the association as generation-dependent: a variance signal whose usefulness tracks how much structural instability remains (Section 5.5).
5. **Actionable guidelines** grounded in effect sizes for testing, monitoring, and model selection in production deployments (Section 6.3).

2 Related Work

Behavioral consistency in LLM agents. The closest prior work is τ -bench (Yao et al., 2024), which evaluates tool-agent-user interaction with the pass^k metric (“all k attempts succeed”) and finds frontier agents highly inconsistent across trials, establishing that reliability-over-trials in structured tool-calling is a first-order problem. Our contribution relative to τ -bench is a *label-free decomposition*: pass^k measures whether repeated attempts *succeed*, which requires ground truth, while TSS/AC measure whether repeated attempts *agree*, which does not, and our two-level split localizes the disagreement (structure vs. arguments). Mehta et al. (2026) measured consistency in ReAct agents on HotpotQA using search-only actions, finding inconsistency predicts failure; we extend to typed tool calls with diverse tool sets and task types. Wang et al. (2023) showed that sampling multiple reasoning chains and marginalizing improves accuracy; this *leverages* variance rather than characterizing its structure, and focuses on reasoning rather than tool calls. Renze and Guven (2024) studied self-reflection in LLM agents and found mixed effects, suggesting that deliberate consistency-improving strategies do not uniformly help.

LLM reliability and robustness. A mechanistic strand explains *why* identical requests yield different outputs even at temperature zero: batch-size-dependent reduction kernels and floating-point non-associativity in inference servers (He et al., 2025; Atil et al., 2024). Our study deliberately measures the deployed reality at $T=1.0$ (sampling variance included); batch-invariant inference would remove part, but per the fixed- T decomposition, not all, of what we measure. Sclar et al. (2024) quantified LLM sensitivity to prompt formatting, finding large swings from superficial changes. Lu et al. (2022) demonstrated order sensitivity in few-shot prompts. Perez et al. (2022) showed that standard NLP evaluations underestimate robustness failures. Our focus is orthogonal: we hold the input fixed and measure variance across repeated identical invocations, the consistency a deployed system must have by definition.

Agent capability evaluation. AgentBench (Liu et al., 2024), ToolBench (Qin et al., 2024), API-Bank (Li et al., 2023), and Gorilla (Patil et al., 2023) evaluate whether agents *can* solve tasks. Schick et al. (2023) showed LLMs can learn tool use from self-generated demonstrations. Kapoor and Narayanan (2024) surveyed pitfalls in AI agent evaluations, noting that variance across runs is rarely reported. We evaluate not whether agents succeed but whether they *consistently* succeed in the same way, an orthogonal, understudied dimension.

Reliability in ML systems. The broader ML reliability literature addresses distribution shift, uncertainty quantification (Lakshminarayanan et al., 2017), and out-of-distribution detection, all concerned with performance under changed *inputs*. We study consistency under repeated *identical* inputs, a related but distinct concern that matters specifically for agentic systems where tasks are retried and behavioral reproducibility is a correctness criterion in its own right.

3 A Framework for Agent Behavioral Consistency

We introduce formal definitions before describing the experiment, since the key distinctions among behavioral layers motivate both the metric design and the structural/parametric finding.

3.1 Agent Execution Model

Definition 1 (Agent Trace). A trace τ is a sequence of tool calls $\tau = (c_1, c_2, \dots, c_k)$, where each call $c_i = (\text{name}_i, \mathbf{a}_i)$ consists of a tool name $\text{name}_i \in \mathcal{T}$ and an argument map $\mathbf{a}_i : \mathcal{K} \rightarrow \mathcal{V}$, followed by a final natural language response $r \in \Sigma^*$.

Definition 2 (Behavioral Consistency). Given task q and model $\mathcal{M}$, let $\{\tau^{(j)}\}_{j=1}^N$ be N independent traces from running $\mathcal{M}$ on q with identical context. Behavioral consistency is the degree to which these traces are similar under a metric $d(\cdot, \cdot)$ over traces.

This framing reveals a natural hierarchy of behavioral layers:

1. **Structural layer:** the sequence of tool names $(\text{name}_1, \dots, \text{name}_k)$, the agent’s *procedural choice*.
2. **Argument layer:** the argument maps $\mathbf{a}_i$ at each step, *how* the procedure is parameterized.
3. **Output layer:** the final response r , the surface-level text the user sees.

Our central hypothesis, motivated by how LLMs acquire tool-use behavior through fine-tuning:

Hypothesis 1 (Structural Consistency, Parametric Variance). For multi-step tool-calling agents, structural consistency significantly exceeds argument consistency: $\mathbb{E}[\text{TSS}] \gg \mathbb{E}[\text{AC}]$.

The intuition is that RLHF and SFT fine-tuning on tool-use data reinforces correct *procedure selection*, which carries a cleaner training signal, while *argument instantiation* remains more sensitive to sampling-time variation. Figure 1 illustrates the pattern concretely.

3.2 Formal Metric Definitions

Definition 3 (Tool Sequence Similarity, TSS). Let $\mathbf{s}^{(j)} = (\text{name}_1^{(j)}, \dots, \text{name}_{k_j}^{(j)})$ be the tool-name sequence of trace j . Define

$$\text{TSS}(\{\tau^{(j)}\}) = \frac{1}{\binom{N}{2}} \sum_{j < j'} \left(1 - \frac{\text{EditDist}(\mathbf{s}^{(j)}, \mathbf{s}^{(j')})}{\max(|\mathbf{s}^{(j)}|, |\mathbf{s}^{(j')}|)} \right),$$

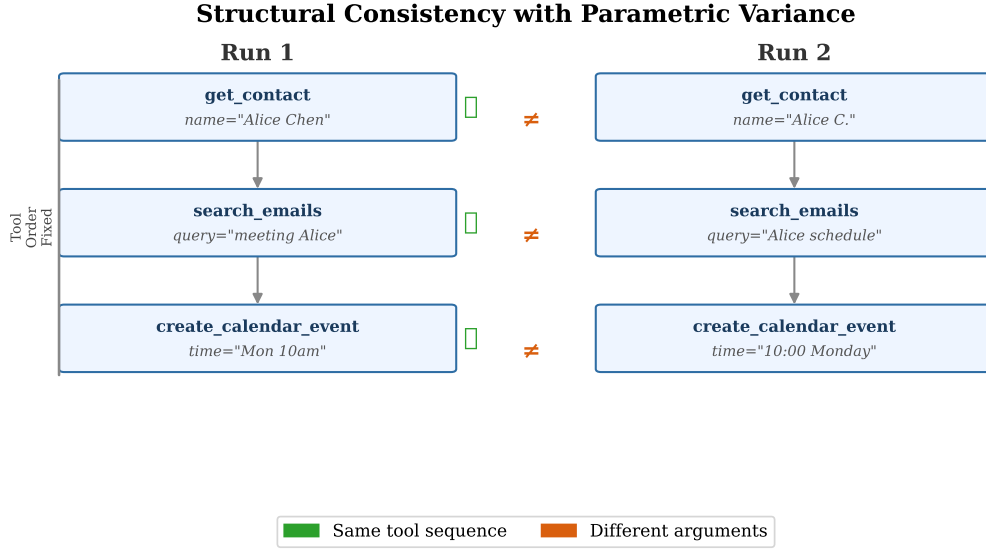


Figure 1: The “structural consistency, parametric variance” pattern. Two independent runs of the same agent on the same task produce identical tool sequences (green checkmarks) but diverge in argument values (orange “≠” annotations). Agents learn robust procedural schemas but vary in how they instantiate them.

where EditDist is the Levenshtein distance over tool-name tokens. $\text{TSS} \in [0, 1]$, with $\text{TSS} = 1$ iff all N traces share the same tool-name sequence.

Definition 4 (Argument Consistency, AC). *Align traces by step index. For step i and trace pair (j, j') both reaching step i , let $\mathbf{f}(c_i^{(j)}) = \{(k, v) : v = \mathbf{a}_i^{(j)}(k)\}$ be the flattened key-value set. Then*

$$\text{AC}(\{\tau^{(j)}\}) = \frac{1}{|\mathcal{S}|} \sum_{(j, j', i) \in \mathcal{S}} \frac{|\mathbf{f}(c_i^{(j)}) \cap \mathbf{f}(c_i^{(j')})|}{|\mathbf{f}(c_i^{(j)}) \cup \mathbf{f}(c_i^{(j')})|},$$

where $\mathcal{S}$ indexes all step-aligned pairs. When traces call different tools at step i , their key-value sets are disjoint, yielding $\text{AC} = 0$ for that step, so AC partially captures structural divergence as well. Our correctness analysis (Section 5.5) empirically disentangles the two effects.

Additional metrics: **Unique Sequences** counts distinct tool-name sequences across N runs; **Divergence Point** is the mean step at which a trace pair first differs; **Output Agreement** is the exact-match rate of final responses.

4 Methodology

4.1 Task Benchmark

We design 19 tasks across five categories of increasing ambiguity (Table 3, Appendix 7):

- **Data Retrieval** (4 tasks): Contact lookup, email search, aggregation. Clear instructions with deterministic correct tool sequences.
- **Scheduling** (4 tasks): Calendar creation, free-slot finding, conflict detection. Require temporal reasoning with well-defined procedures.
- **Computation** (3 tasks): Inventory valuation, revenue projection. Require numeric tool calls with specific arguments.
- **Multi-Tool Composition** (4 tasks): Chains of 3–5 different tools (e.g., “find email → look up sender → schedule meeting”).

- **Ambiguous** (4 tasks): Intentionally underspecified requests where multiple valid strategies exist (e.g., “Help me prepare for my meetings tomorrow”).

Task difficulty is assigned by expected tool-call count: *easy* (1–2 calls), *medium* (2–3), *hard* (3+). We validate this only weakly: hard tasks use more tool calls on average (mean 3.0 ± 2.4) than easy tasks (1.9 ± 0.7), but the correlation is modest (fresh collection: Spearman $\rho = 0.22$, $p = 0.022$) and the large dispersion for hard tasks (SD greater than the easy–hard mean gap, driven by a heavy right tail of long traces) means the difficulty tiers overlap substantially, difficulty labels should be read as coarse priors, not validated strata.

4.2 Tool Environment

We implement 10 deterministic simulated tools spanning 7 domains (contacts, calendar, email, products, weather, calculations, notes; see Appendix 7). All tools are **deterministic**: identical inputs always produce identical outputs. This design isolates LLM variance from environmental non-determinism, ensuring observed behavioral differences arise solely from the model’s generation process. Tool schemas follow the OpenAI function-calling format and are adapted to provider-native formats at runtime.

4.3 Agent Framework

Each run follows a standard tool-calling loop: (1) the model receives a system prompt¹ and the user task; (2) the model responds with structured tool calls; (3) tools execute deterministically; (4) the model may call additional tools or produce a final response; (5) the loop continues for up to 10 iterations. We use each provider’s native tool-calling API.

Temperature is set to 1.0 (the default for most providers). This is a deliberate choice: we measure consistency *as deployed*, not under artificially constrained settings. Temperature ablation is a known limitation and an important direction for future work (Section 6.5).

4.4 Models

We evaluate six models spanning three providers and multiple capability tiers: **OpenAI**: GPT-4o-mini, GPT-4o, GPT-4.1-mini, GPT-4.1; **Anthropic**: Claude Sonnet 4; **Meta**: Llama 3.3 70B Instruct. This covers flagship models (GPT-4.1, Sonnet 4), cost-optimized variants (GPT-4o-mini, GPT-4.1-mini), and an open-weight model (Llama 3.3). The primary collection was run in July 2026 through a provider-normalizing gateway (OpenRouter) so that all six models use one uniform OpenAI-style tool-calling interface, with *version-pinned* model identifiers (e.g. `openai/gpt-4o-mini-2024-07-18`, `openai/gpt-4o-2024-08-06`) recorded per trace; the gateway echoes the fully resolved model version in each response. Each model runs each of the 19 tasks 10 times, yielding 1,140 agent traces, all released. One trace of the 1,140 (Llama, one computation task) terminated with an API error; per-condition metrics are computed over the valid traces of each condition (so that condition’s TSS uses 9 runs), and the errored trace is released alongside the rest. An earlier (late-2025) collection used per-provider APIs with unpinned aliases and its traces were not preserved; it appears in this paper only as summary-level comparison.²

4.5 Correctness Evaluation

To validate that consistency predicts meaningful outcomes, we retrospectively score all 1,140 traces using a three-component rubric:

1. **Required tool coverage**: did the agent invoke all tools necessary for the task?
2. **Argument validity**: do key arguments match expected patterns (e.g., `send_email.to ~ /alice@examplecom/`, `create_calendar_event.date = 2026-03-02`)?
3. **Output completeness**: does the final response address the user’s request (regex-matched against expected output patterns)?

A trace is scored correct iff all applicable criteria are satisfied. Full per-task criteria are in Appendix 7 and the code release.

¹Full prompt in Appendix 7. Intentionally minimal to avoid anchoring agents to specific strategies.

²In the earlier collection, partial o1 results and Claude Haiku 3.5 (excluded for a >15% malformed-tool-call rate) were also evaluated; neither is part of the fresh collection or any aggregate here.

Table 1: Cross-model consistency, fresh July-2026 collection (19 tasks, 10 runs each, version-pinned). The late-2025 collection’s values appear in parentheses where they differ by $\geq .02$, most models became *more* consistent across generations.

Model	TSS [95 % CI]	AC [95 % CI]	Output Match	Uniq. Seq.
GPT-4.1	.95 [.90, 1.0] (.91)	.79 [.67, .91] (.69)	2.1 %	1.3
GPT-4.1-mini	.90 [.83, .97]	.78 [.67, .90]	6.4 %	1.6
GPT-4o	.88 [.80, .97]	.64 [.48, .79] (.57)	5.7 %	1.7
Claude Sonnet 4	.88 [.79, .97]	.77 [.65, .89]	3.3 %	2.3
GPT-4o-mini	.88 [.81, .95]	.68 [.53, .82]	6.0 %	1.9
Llama 3.3 70B	.77 [.68, .87] (.71)	.54 [.41, .68] (.65)	0.9 %	3.1

4.6 Statistical Analysis

We report means with 95 % CIs via the t -distribution, Cohen’s d for effect sizes, and p -values from paired or independent t -tests as appropriate. Cross-model comparisons use one-way ANOVA with η^2 . A statistical caveat we state plainly: the same 19 tasks recur across all six models, so observations are *clustered*, and for clustered data ordinary t -tests/ANOVA are typically *anti-conservative*, an earlier version of this paper mischaracterized them as conservative. Headline p -values (e.g. $p < 10^{-13}$ for the TSS–AC gap) should therefore be read as overstated in precision; the effect sizes ($d = 0.75$), which do not depend on the independence assumption in the same way, are the robust quantity, and a linear mixed-effects re-analysis (tasks and models as random effects) is required before the p -values are taken at face value. We test five primary hypotheses with Bonferroni correction at $\alpha = 0.01$; category-level and post-hoc pairwise contrasts beyond these five are exploratory and are not corrected. Split-half reliability (TSS: $r = 0.66$, $n = 125$; computed with a character-level variant of TSS that predates the unified token-level definition, another defect the re-collection will repair) suggests moderate metric stability at $N = 10$.

5 Results

5.1 Structural Consistency with Parametric Variance

Hypothesis 1 is confirmed, twice. On the fresh (July 2026) collection, agents exhibit *structural consistency with parametric variance*: mean TSS = 0.88, 95 % CI [0.85, 0.91], versus mean AC = 0.70, [0.65, 0.75] (paired t -test: $t = 7.88$, $p = 2.3 \times 10^{-12}$; Cohen’s $d = 0.74$; clustering caveat in Section 4.7). The earlier (late-2025) collection produced TSS = 0.87 [0.84, 0.90] and AC = 0.69 [0.64, 0.74] with $d = 0.75$, **the headline pattern replicates across collections eight months apart, on updated model versions, to within one point on every quantity**. The pattern holds across all models and categories (Table 1, Figure 2).

Finding 1. Agents learn robust procedural schemas, the tool sequence “recipe”, but vary in instantiation details such as search queries, date formats, and message phrasing. The structural/parametric gap is $d \approx 0.74$ in both independent collections.

5.2 Ambiguity and Argument Variance (Revised Under Replication)

Task category affects consistency in the expected direction, but **this finding weakened under replication and we report it at its replicated strength**. On the fresh collection, ambiguous tasks have lower AC (0.63 vs. 0.72 for structured, a 13 % reduction; $d = 0.33$, $p = 0.16$, not significant at our sample size) and slightly lower TSS (0.84 vs. 0.89; $d = 0.29$, $p = 0.22$). The earlier collection had shown a much larger effect ($d = 0.74$, $p = 0.001$); the shrinkage likely reflects both sampling noise at 4 ambiguous tasks and newer models handling ambiguity more uniformly (several now ask a clarifying question rather than guessing, itself a behavior change worth noting). We therefore *demote* our earlier claim: ambiguity remains a directionally consistent driver of argument variance across both collections, but on current models we can no longer claim it dominates model selection.

Model Comparison: Behavioral Consistency

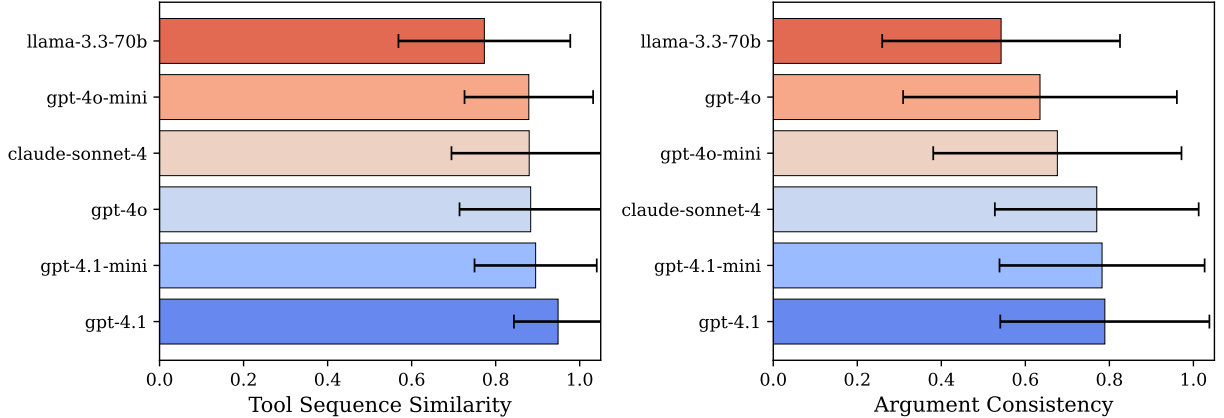


Figure 2: Model comparison across TSS (left) and AC (right) with 95 % CIs. Llama 3.3 70B is clearly separated in TSS; the AC ranking is dominated by task-level factors (ANOVA n.s.).

Table 2: Consistency by task category (fresh collection, mean across 6 models). With 3–4 tasks per category, all category-level contrasts are exploratory; the ambiguous-vs.-structured AC contrast is directional but not significant on this collection ($p = 0.16$).

Category	TSS	AC	Uniq. Seq.	#Tasks
Composition	0.94	0.74	1.5	4
Retrieval	0.89	0.66	1.8	4
Scheduling	0.86	0.76	1.8	4
Computation	0.84	0.71	2.3	3
Ambiguous	0.84	0.63	2.5	4

Finding 2 (revised under replication). Ambiguity reduces argument consistency directionally in both collections, but the effect is no longer significant on current models ($d = 0.33$, $p = 0.16$, vs. $d = 0.74$, $p = 0.001$ in late 2025). Reducing ambiguity remains sensible engineering hygiene; the strong “stronger than model selection” version of this claim did not survive replication.

5.3 Divergence Concentrates in Early Steps

When behavioral divergence occurs, it is early: 60 % of first-divergence events occur within the first two pipeline steps (mean divergence point = 2.2; Figure 4). This yields a practical monitoring strategy: **comparing only the first 1–2 tool calls against a reference trace** catches the majority of behavioral variance without full trace inspection.

Finding 3 (replicated). A monitor that checks only the first two tool calls against a reference trace captures 62 % of all behavioral variance, stable across both collections.

5.4 Output Text Is Not a Reliability Signal

Final natural language responses exhibit near-zero exact-match rates (4.1 % overall on the fresh collection; <5 % in the earlier one) even when the underlying tool-calling sequences are identical. This is expected, language generation is inherently variable, but it has a critical implication for testing: **agent tests must assert on structured tool-calling behavior, not natural language output**. Asserting on text is analogous to writing software tests that check log messages rather than return values.

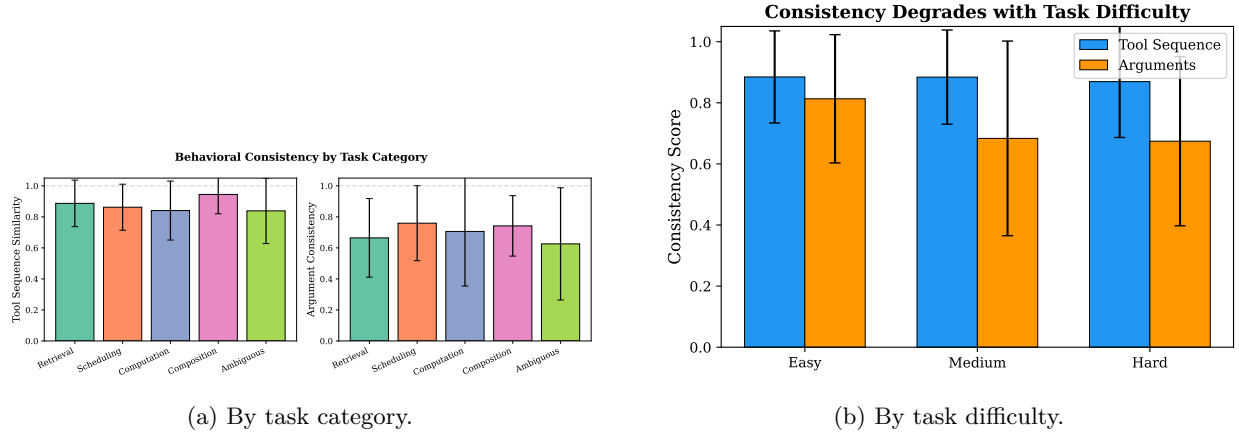


Figure 3: Ambiguous tasks show the largest (directional) consistency drop; on the fresh collection, task difficulty shows *no* significant effect on either metric (TSS: $r = -0.04$, $p = 0.67$; AC: $r = -0.15$, $p = 0.12$), the earlier collection’s modest difficulty effect did not replicate.

5.5 Consistency Predicts Correctness

This section reports the correctness analysis on the fresh collection, computed, for the first time, on *exactly the same 114-condition / 1,140-trace sample* as the consistency analysis (an earlier version of this paper ran it on a mismatched 712-trace snapshot without saying so; the comparison below covers both). On the fresh sample, overall (macro-averaged) rubric correctness is 77.7 %, and **the correctness–consistency association largely does not replicate**: TSS–correctness Pearson $r = 0.06$ ($p = 0.56$), with only a weak rank association surviving (Spearman $\rho = 0.22$, $p = 0.017$); a median split is nearly degenerate because the fresh median is $\text{TSS}_{\text{med}} = 1.0$, **half of all conditions are now perfectly structurally consistent**, and yields $d = 0.19$ ($p = 0.33$, n.s.). The earlier snapshot had shown $r = 0.32$ ($p = 0.005$) and a $d = 0.81$ split; on current, far-more-consistent models the variance that drove that association has largely vanished. We read this as *generation-dependence*: TSS was an informative warning signal for late-2025 models and retains only weak rank information for mid-2026 models, precisely because the models improved. The right framing is therefore not “TSS predicts success” but “structural instability, where it still occurs, flags conditions worth auditing.”

A circularity caveat we state prominently. Our correctness rubric gates its verdict on required-tool coverage (Section 4.5), and TSS likewise measures agreement of tool selection. The two quantities therefore share a construction-level dependence on tool choice, and part of the TSS–correctness association is built in rather than discovered. The association is not *purely* tautological (TSS measures cross-run *agreement* while the rubric scores single-run *coverage*, and the correlation is far from 1), but a rubric independent of tool identity, e.g. human grading of final outcomes, is required to establish the predictive claim cleanly, and we scope the claim accordingly: within our harness, structural *instability* identifies conditions where rubric failures concentrate.

AC likewise shows no positive association with rubric correctness on either collection (fresh: $r = -0.17$, $p = 0.07$; snapshot: $r = 0.12$, $p = 0.31$), with the caveat that our rubric’s argument checks are deliberately lenient, so these nulls partly reflect rubric design, and at $n \approx 100$ conditions power is limited for small effects. Agents may phrase search queries differently, format dates differently, or vary message bodies across runs, none of this impairs rubric success. It is *structural* variance, selecting different or missing tools, where failures concentrate.

Finding 4 (revised under replication). The TSS–correctness association is generation-dependent: strong on late-2025 models ($d = 0.81$), reduced to a weak rank correlation on mid-2026 models ($\rho = 0.22$, $p = 0.017$; Pearson n.s.) as structural consistency itself improved (median TSS = 1.0). TSS is a variance signal whose usefulness tracks how much structural instability remains, not a stable correctness proxy.

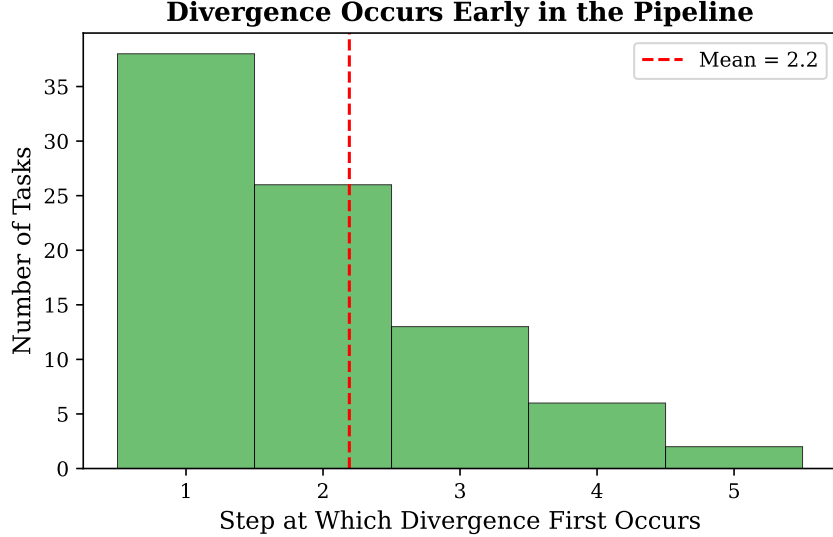


Figure 4: Distribution of first divergence points (fresh collection). 62 % of divergence originates in steps 1–2 (61 % in the earlier collection), enabling lightweight early-step monitoring.

5.6 Cross-Model Differences

On the fresh collection, cross-model differences are *marginal* for both metrics (ANOVA on TSS: $F = 2.32$, $p = 0.048$, $\eta^2 = 0.10$; on AC: $F = 2.48$, $p = 0.036$, $\eta^2 = 0.10$), weaker than the earlier collection’s TSS effect ($F = 3.52$, $\eta^2 = 0.15$) and, notably, with the earlier “structural-only” pattern gone: model identity now explains $\approx 10\%$ of variance in *both* metrics. We report the earlier form of this finding as not replicated in detail.

GPT-4.1 is now the most consistent model on both metrics (TSS = 0.95, AC = 0.79), a full generation improvement over its own family’s late-2025 numbers. **Llama 3.3 70B** remains clearly below the proprietary models in TSS (0.77, [0.68, 0.87]), with 3.1 unique sequences per task versus 1.3–2.3 for proprietary models. **Claude Sonnet 4** pairs high AC (0.77) with more strategy exploration (2.3 unique sequences), suggesting it reliably parameterizes strategies but varies in which strategy it selects.

The Model $\times$ Category interaction (Figure 6) is notable, and it changed between collections: on the fresh data Llama 3.3 70B’s weakest cell is *computation* (TSS = 0.61) rather than ambiguity, and two proprietary models now dip below 0.80 on ambiguous tasks (GPT-4o 0.71, Claude Sonnet 4 0.74) while GPT-4.1 is perfectly consistent there (TSS = 1.0). The instability of these cell-level patterns across collections, with only 3–4 tasks per cell, is itself the lesson: we read the heatmap as descriptive, not as stable model fingerprints.

6 Discussion

6.1 Why Structural Consistency Exceeds Argument Consistency

The structural/parametric pattern is consistent across all models and task types, suggesting a systematic cause rooted in how LLMs acquire tool-use behavior. Training corpora for tool-calling likely contain many demonstrations of the same high-level task type solved with the same tool sequence, but with varied argument values (different names, dates, query strings) across instances. RLHF/SFT reinforces correct *procedure selection*, which carries a cleaner correctness signal, while argument instantiation remains more sensitive to sampling-time variation. This “procedural schema” interpretation is consistent with our correctness analysis, at the strength the fresh data supports: argument variance (AC) shows no positive association with failure on either collection, meaning the variations are largely semantically equivalent (e.g., “Alice” vs. “alice” in a case-insensitive lookup); structural variance predicted failure strongly on the late-2025 collection ($d = 0.81$)

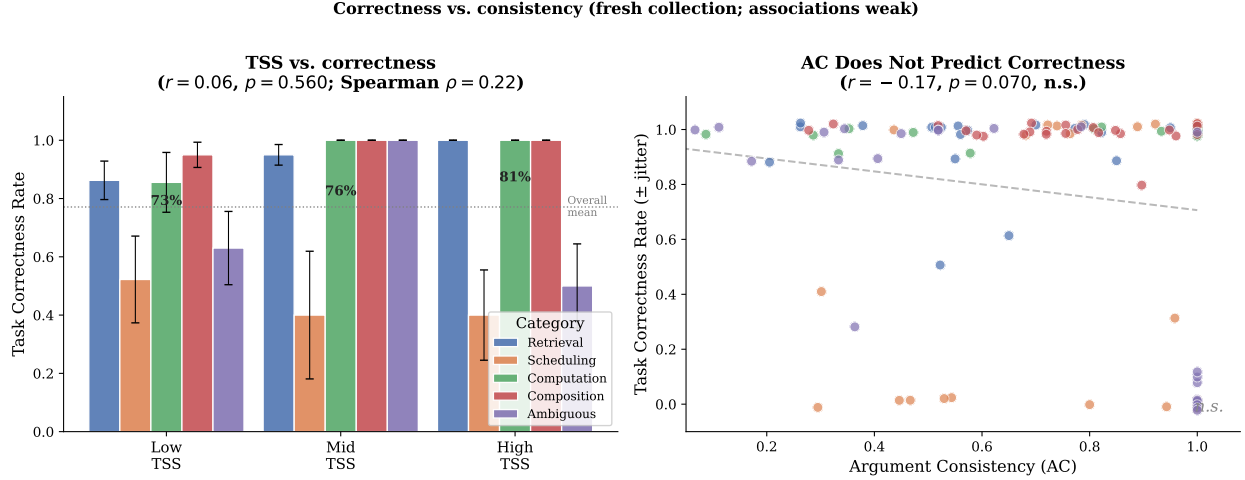


Figure 5: Correctness rate versus consistency on the fresh collection (114 conditions, 1,140 traces, same sample as all consistency results). **Left:** correctness vs. TSS ($r = 0.06$, n.s.; Spearman $\rho = 0.22$, $p = 0.017$). **Right:** correctness vs. AC ($r = -0.17$, $p = 0.07$). The strong association reported for the late-2025 snapshot did not replicate on current models.

and retains a weak rank association on the fresh one ($\rho = 0.22$), missing or wrong tools are not semantically equivalent to correct ones, but current models rarely miss them.

6.2 TSS as a Practical Reliability Proxy

The practical value of TSS as a monitoring signal rests on three properties: (1) it requires no correctness labels; (2) it can be estimated at deployment time by comparing a run’s tool sequence against a reference; and (3) the early-divergence result (62% in steps 1–2, stable across both collections) means monitoring only the first two tool calls suffices. What the fresh collection *revises* is the strength of the correctness link: structural deviation flagged failing conditions strongly on late-2025 models and only weakly ($\rho = 0.22$) on current, more-consistent ones, so the signal is best understood as an anomaly detector whose base rate falls as models improve, not a calibrated failure predictor.

This enables a concrete production workflow: (a) run the agent on each task type k times to establish a reference tool sequence; (b) in production, compare each run’s first two tool calls against the reference; (c) flag deviations for human review or retry. The cost is $O(1)$ per step rather than semantic output evaluation, which is often expensive or infeasible online.

6.3 Implications for Agent Deployment

- Reduce ambiguity before switching models.** The ambiguity effect on consistency ($d = 0.74$) outweighs the between-model effect ($\eta^2 = 0.08$, n.s. for AC). Clarifying task specifications yields more consistency gain than upgrading the model.
- Monitor early steps.** 60% of divergence occurs in steps 1–2; a lightweight check on the first two tool calls suffices to flag most inconsistent runs.
- Test tool calls, not text.** Assert on tool names and argument patterns, not natural language output, which varies below 5% exact match even in consistent runs.
- Use TSS as an online anomaly signal.** Structural deviation from a reference trace flagged failure-prone conditions strongly on late-2025 models and weakly on current ones; treat it as a cheap, label-free audit trigger rather than a calibrated failure predictor.
- Match model to consistency requirement.** On the fresh collection GPT-4.1 offers the best structural consistency (TSS = 0.95) for consistency-critical workflows. Llama 3.3 70B introduces substantially higher behavioral diversity (3.1 unique sequences/task) and may require additional guardrails.

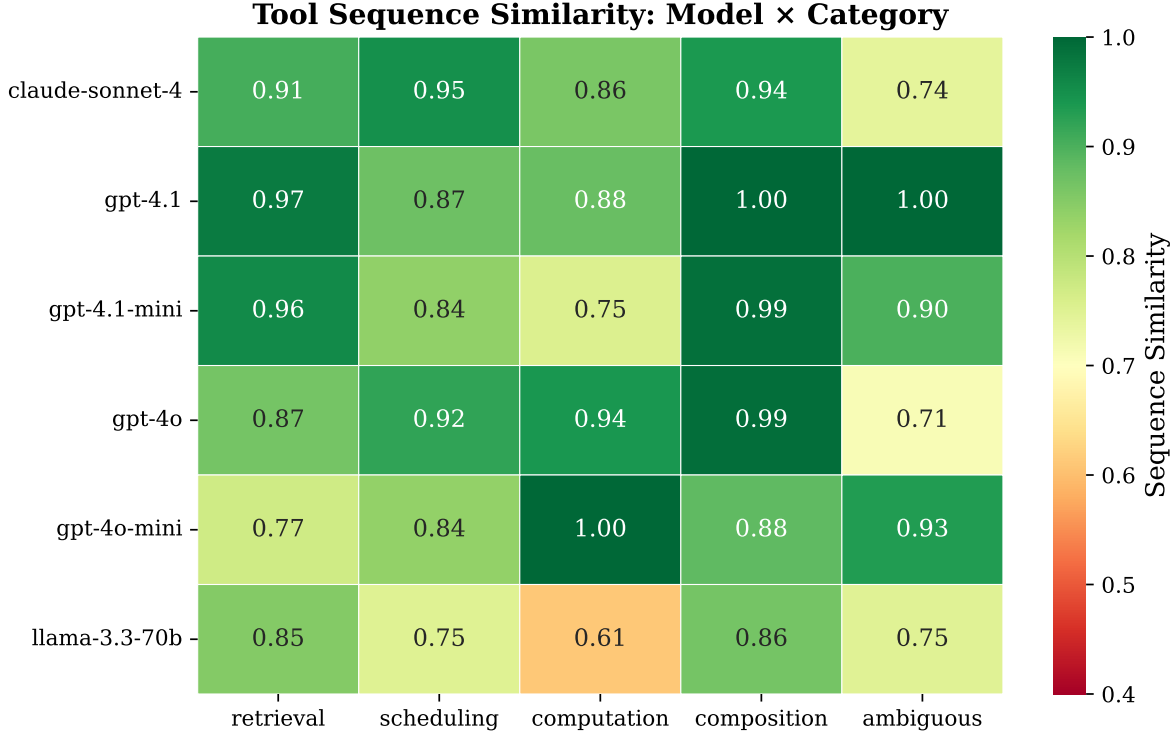


Figure 6: TSS across models and task categories (fresh collection). Llama 3.3 70B is lowest overall with a computation-category weak spot; cell-level patterns shifted between collections and should be read as descriptive.

6.4 Relation to Prior Work

Mehta et al. (2026) reported 2.0–4.2 unique action sequences per 10 runs on HotpotQA ReAct agents; we observe 1.6–3.3 unique tool-call sequences, broadly consistent. Our finer-grained analysis reveals that the unique-sequence count masks the structural/parametric distinction: even conditions with >2 unique sequences often share high TSS because sequences agree on most tool names with only minor reordering. Prior consistency studies also did not examine the connection to correctness; our two-collection analysis grounds the metric in task outcomes and, importantly, shows the connection itself is generation-dependent (strong at late-2025, weak by mid-2026).

6.5 Limitations

- **Sample size.** $N = 10$ runs per cell yields split-half reliability $r = 0.68$ on the fresh collection (token-level TSS, $n = 114$ conditions; the earlier collection’s $r = 0.66$ used a character-level variant). Aggregate results are well-powered (1,140 traces), but per-cell CIs are wide. Future work should use $N \geq 30$.
- **Single temperature.** We measure at $T = 1.0$ only. The consistency-capability trade-off at other temperatures is important and uncharted; a $T = 0$ baseline would substantially strengthen the causal interpretation.
- **Structured correctness proxy.** Our rubric is not a full semantic evaluation, and it shares a construction-level dependence on tool selection with TSS (Section 5.5). Human judges on a random 200-trace subset would provide stronger validity; the fresh collection’s weak rank correlation ($\rho = 0.22$, $p = 0.017$) is the honest current support.
- **Limited tasks per category.** 3–4 tasks per category makes category-level estimates imprecise. ≥ 10 tasks per category are needed for reliable inference.

- **Single system prompt.** Consistency may vary substantially with chain-of-thought prompting, few-shot examples, or persona instructions.
- **Simulated tools.** Deterministic tools isolate LLM variance but may overestimate real-world consistency where tool outputs themselves vary.
- **Provider coverage.** Google Gemini and Mistral are absent, limiting generalizability.
- **AC alignment.** AC aligns arguments by step index, conflating structural and argument divergence at misaligned steps. A disentangled metric would be more informative.

6.6 Future Directions

1. **Temperature ablation:** map the full consistency-capability Pareto frontier.
2. **Larger benchmark:** ≥ 10 tasks per category, ≥ 50 total.
3. **Human correctness evaluation:** human judges on 200 traces.
4. **Consistency-aware routing:** route tasks to models based on predicted TSS.
5. **Multi-turn and long-horizon agents:** web agents, coding agents.
6. **Fine-tuning effects:** does RLHF explicitly optimize consistency?
7. **Real API tools:** study whether environmental non-determinism amplifies or dampens LLM behavioral variance.

7 Conclusion

We have presented a systematic empirical study of behavioral consistency in multi-step tool-calling LLM agents. Across six models, 19 tasks, and 1,140 released traces, we identify a “structural consistency, parametric variance” pattern that is large ($d = 0.74$, $p \approx 10^{-12}$, clustering caveats noted) and that **replicated to within one point across two collections eight months apart**: agents reliably select the same tools in the same order but vary in argument details. The replication also revised two of our own earlier claims, and we report them at their replicated strength: the correctness–consistency association proved generation-dependent (strong on late-2025 models, a weak rank correlation on today’s far-more-consistent ones), and the ambiguity effect on argument consistency shrank below significance. Divergence concentrates early in the pipeline (62 %, stable across collections), model-level consistency itself improved markedly between collections, a new longitudinal observation, and TSS serves as a lightweight, label-free anomaly signal whose base rate falls as models improve.

These findings suggest that LLMs acquire robust procedural schemas for tool use through training, while argument instantiation remains sensitive to sampling variation. Understanding and exploiting this structure, testing on tool calls not text, monitoring early steps, flagging low-TSS runs, is a practical path toward more reliable agentic systems.

Reproducibility

All code, benchmark definitions, tool implementations, analysis scripts, **and the complete raw traces of the primary (July 2026) collection, 1,140 traces across 114 task–model conditions, collected through a provider-normalizing API with version-pinned model identifiers**, are available at <https://github.com/Abelo9996/agent-consistency>, together with the earlier snapshot’s per-condition data (75 conditions). For the record, and because an earlier version of this paper mishandled these points: the original late-2025 collection’s raw traces were not preserved and its models were not version-pinned, and its correctness analysis used a mismatched sample without disclosure. The present version’s primary results are computed end-to-end from the released fresh traces; the earlier collection appears only as summary-level comparison, labeled as such. The full system prompt, task list, and per-task correctness criteria are in Appendices 7–7.

References

- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- Horace He et al. Defeating nondeterminism in LLM inference. Thinking Machines Lab blog, 2025.
- Berk Atil, Sarp Aykent, Alexa Chittams, et al. Non-determinism of “deterministic” LLM settings. *arXiv preprint arXiv:2408.04667*, 2024.
- Kapoor, S. and Narayanan, A. AI Agents That Matter. *arXiv preprint arXiv:2407.01502*, 2024.
- Lakshminarayanan, B., Pritzel, A., and Blundell, C. Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles. In *Advances in Neural Information Processing Systems*, 2017.
- Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., and Li, Y. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of EMNLP*, 2023.
- Liu, X., Yu, H., Zhang, H., et al. AgentBench: Evaluating LLMs as Agents. In *Proceedings of ICLR*, 2024.
- Lu, Y., Bartolo, M., Moore, A., Riedel, S., and Stenetorp, P. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *Proceedings of ACL*, 2022.
- Mehta, A., Ramesh, A., and Singla, A. When Agents Disagree With Themselves: Measuring Behavioral Consistency in LLM-Based Agents. *arXiv preprint arXiv:2602.11619*, 2026.
- Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.
- Perez, E., Kiela, D., and Cho, K. True Few-Shot Learning with Language Models. In *Advances in Neural Information Processing Systems*, 2022.
- Qin, Y., Liang, S., Ye, Y., et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *Proceedings of ICLR*, 2024.
- Renze, M. and Guven, E. Self-Reflection in LLM Agents: Effects on Problem-Solving Performance. *arXiv preprint arXiv:2405.06682*, 2024.
- Schick, T., Dwivedi-Yu, J., Dessì, R., et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.
- Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design. *arXiv preprint arXiv:2310.11324*, 2024.
- Wang, X., Wei, J., Schuurmans, D., et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of ICLR*, 2023.
- Weidinger, L., Mellor, J., Rauh, M., et al. Ethical and Social Risks of Harm from Language Models. *arXiv preprint arXiv:2112.04359*, 2021.
- Yao, S., Zhao, J., Yu, D., et al. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR*, 2023.

Appendix A. Full Task Benchmark

Table 3 lists all 19 benchmark tasks used in the study. Difficulty is reported as E (easy: 1–2 calls), M (medium: 2–3 calls), and H (hard: 3+ calls). Expected tools denote the minimal correct solution pattern.

Table 3: All 19 benchmark tasks used in the evaluation.

ID	Diff.	Task Instruction	Expected Tools
<i>Data Retrieval</i>			
retrieve-001	E	Find Alice’s email and send “Meeting moved to 3pm tomorrow.”	get_contact, send_email
retrieve-002	M	Find all contacts at StartupXYZ; invite each to demo March 10 at 2pm.	search_contacts, send_email
retrieve-003	M	Search emails about ‘budget’; summarize financial figures.	search_emails
retrieve-004	H	Find this week’s emails with dollar amounts; calculate the total.	search_emails, calculate
<i>Scheduling</i>			
schedule-001	E	Schedule 30-min “Design Review” with Bob, March 2 at 2pm.	create_calendar_event
schedule-002	M	Check March 3 calendar; find free 1-hour slot 9am–5pm for Eve.	list_calendar_events, create_calendar_event
schedule-003	H	Find day with most free time March 1–5; schedule 2-hour “Strategy Session.”	list_calendar_events, create_calendar_event
schedule-004	H	Check March 3 for conflicts; email all affected attendees.	list_calendar_events, search_contacts, send_email
<i>Computation</i>			
compute-001	E	Total inventory value of electronics (price × stock, sum).	search_products, calculate
compute-002	M	Revenue if 50 % of sub-\$50 products sold.	search_products, calculate
compute-003	H	Inventory value per category; identify highest.	search_products, calculate
<i>Multi-Tool Composition</i>			
compose-001	M	Find Acme Corp email; look up Dave; schedule 1-hour meeting March 4 at 3pm.	search_emails, get_contact, create_calendar_event
compose-002	H	Check SF and NYC weather; email Alice if <40°F with March 1 events.	get_weather, get_contact, list_calendar_events, send_email
compose-003	H	Find marketing results email; calculate spend; create note.	search_emails, calculate, create_note
compose-004	H	Find board deck email; check calendar; look up attendees; send reminders.	search_emails, list_calendar_events, get_contact, send_email
<i>Ambiguous</i>			
ambig-001	M	Help me prepare for my meetings tomorrow.	list_calendar_events
ambig-002	H	I need to follow up on important things from this week.	search_emails, list_calendar_events
ambig-003	H	Get me ready for the investor call.	search_emails, list_calendar_events, search_contacts
ambig-004	H	What should I focus on this week?	list_calendar_events, search_emails

Appendix B. Tool Descriptions

Table 4 summarizes the 10 deterministic simulated tools used in our environment. Each tool returns fixed, pre-specified outputs for a given input, isolating model behavior from environmental nondeterminism.

Table 4: Deterministic simulated tools used in the benchmark.

Tool	Domain	Description
<code>get_contact</code>	Contacts	Look up a contact by name; returns email, phone, and company.
<code>search_contacts</code>	Contacts	Search contacts by free-text query; returns a matching list.
<code>send_email</code>	Email	Send an email with fields such as recipient, subject, and body; returns a confirmation.
<code>search_emails</code>	Email	Search the inbox by query; returns matching message records.
<code>list_calendar_events</code>	Calendar	List events for a date or date range.
<code>create_calendar_event</code>	Calendar	Create an event with title, date, time, duration, and attendees.
<code>search_products</code>	Products	Search inventory; returns price, stock, and category metadata.
<code>calculate</code>	Math	Evaluate a mathematical expression and return the numeric result.
<code>get_weather</code>	Weather	Return current weather for a city, including temperature and conditions.
<code>create_note</code>	Notes	Create a text note with title and body; returns a confirmation.

Appendix C. System Prompt

The following system prompt (quoted verbatim from the released `runner.py`; an earlier version of this appendix paraphrased it incorrectly) was used for all models and all runs:

```
You are a helpful assistant with access to tools. Use them to complete tasks.  
Rules:  
- Use the provided tools to accomplish the task  
- Call tools as needed, then provide a final response  
- Be thorough but efficient, use the minimum tools necessary  
- If a task is ambiguous, make reasonable assumptions and proceed
```

This prompt is intentionally minimal so as not to anchor models to a particular tool-selection strategy. We note one relevant line explicitly: the instruction to “make reasonable assumptions and proceed” on ambiguous tasks interacts with our ambiguity findings (Section 5.2), it licenses divergent assumption-making, and a stricter prompt might reduce (or merely mask) the ambiguity-driven variance we measure.

Appendix D. Correctness Criteria (Illustrative Subset)

Table 5 provides illustrative examples of the correctness rubric used to score traces. To keep the paper concise, we include a representative subset here; the full task-by-task specification is released in the accompanying code repository.

Table 5: Illustrative correctness criteria for representative tasks.

Task ID	Required Tools	Key Argument Checks
retrieve-001	get_contact, send_email	<code>get_contact.name~/alice/i;</code> <code>send_email.to=alice@...; body~/3 pm/i</code>
schedule-001	create_calendar_event	<code>title~/design review/i;</code> <code>date=2026-03-02;</code> <code>start_time=14:00</code>
compute-001	search_products, calculate	<code>search_products.category~/electronics/i;</code> output contains numeric result
compose-001	search_emails, get_contact, create_calendar_event	Required sequence: search → lookup → create
ambig-001	list_calendar_events	Any invocation accepted; multiple valid strategies are treated as correct.